

Backend Workflow Documentation

Nandini Bhimineni

yoganandini9@gmail.com

Overview

This document explains how our application processes client requests. It describes how requests are received, validated, processed, stored, monitored, and responded to within the backend system.

1. System Architecture Overview

Our backend architecture follows a layered design approach to ensure scalability, maintainability, performance, and reliability.

Core Components

Component	Purpose
API Layer	Receives and routes client requests
Authentication Service	Verifies user identity and access permissions
Business Logic Layer	Executes application-specific operations
Database Layer	Stores and retrieves application data
Cache Layer	Improves response speed and performance
Queue/Worker System	Processes long-running background tasks
Notification Service	Sends email, SMS, and push notifications
Monitoring & Logging	Tracks system performance and errors

2. Backend Request Workflow

Step 1: Client Request

The workflow begins when a client application sends a request to the backend API.

Examples

- User login,File upload,Data retrieval

Step 2: API Gateway / Routing

The API gateway is responsible for receiving incoming requests and routing them to the appropriate backend services.

Responsibilities

- Request forwarding
- Endpoint routing
- Rate limiting
- Request filtering
- Load balancing

Step 3: Authentication & Authorization

The authentication module validates user identity and verifies access permissions before processing requests.

Verification Includes

- User authentication
- Access token validation
- Session verification
- Permission checks

Common Technologies

- JWT Authentication
- OAuth 2.0
- Session-based Authentication

Step 4: Request Validation

The validation layer ensures all incoming requests meet required business and security standards.

Validation Checks

- Required field verification
- Data format validation
- Business rule enforcement
- Input sanitization

Examples

- Email format validation, Password policy verification, Request payload validation

Step 5: Business Logic Processing

The business logic layer handles the core functionality of the application.

Example Operations

- Order creation, Payment processing
- Inventory updates
- Report generation

Step 6: Database Operations

The backend interacts with databases to store, retrieve, and update application data.

Database Responsibilities

- Creating records
- Retrieving data
- Updating transactions
- Maintaining data consistency

Supported Databases

- PostgreSQL
- MySQL
- MongoDB

Step 7: External Service Integration

The backend communicates with third-party services and external platforms when required.

Integrated Services

- Payment gateways
- Email providers
- Cloud storage platforms
- Third-party APIs

Key Considerations

- Retry mechanisms

- Timeout management
- Error handling
- Secure API communication

Step 8: Asynchronous Processing

Resource-intensive and long-running tasks are processed asynchronously using queues and worker services.

Examples

- Email delivery
- Report generation
- Image processing
- Notification dispatching

Queue Technologies

- RabbitMQ
- Apache Kafka
- Amazon SQS

Step 9: Logging & Monitoring

The monitoring infrastructure continuously tracks system activity, performance, and errors.

Monitoring Includes

- Application performance tracking
- API response monitoring
- System error detection
- Security event logging

Monitoring Tools

- Prometheus
- Grafana
- ELK Stack
- Datadog

Step 10: Response Delivery

After request processing is completed, the backend returns an appropriate response to the client.

Response Types

- Successful responses
- Validation errors
- Authorization failures
- System exceptions

3. Example Workflow: Order Processing

Customer Places Order



API Receives Request



Authentication Validation



Cart & Inventory Verification



Payment Processing



Order Creation in Database



Notification Triggered



Warehouse Processing Initiated



Response Sent to Client

4. Security Considerations

The backend workflow incorporates multiple security mechanisms to protect application data and services.

Security Controls

- HTTPS encryption
- Token-based authentication
- Input sanitization
- Role-based access control
- Secure secret management
- Audit logging

5. Scalability Strategy

The backend architecture is designed to support system growth and increasing traffic demands.

Scalability Features

- Horizontal scaling
- Stateless services
- Distributed caching
- Load balancing
- Containerized deployment
- Auto-scaling infrastructure

6. Deployment Workflow

CI/CD Pipeline

1. Code commit
2. Automated testing
3. Build generation
4. Security scanning
5. Deployment to staging
6. Production deployment

Deployment Tools

- GitHub Actions
- Jenkins
- Docker

- Kubernetes

7. Conclusion

The backend workflow is designed to provide:

- High availability
- Scalability
- Secure request processing
- Reliable service integrations
- Maintainable system architecture

This architecture ensures efficient request handling while maintaining application performance, operational reliability, and security across all backend services.